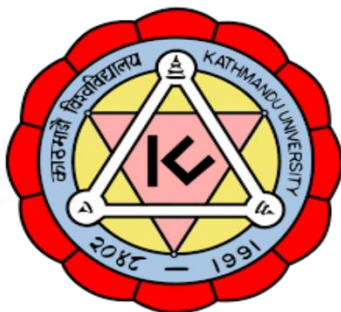


Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre



A Project Report
On
“eYatra Ride Sharing App”
[Code No: COMP 232]
(For partial fulfillment of Year II / Semester II in Computer Engineering)

Submitted by:
Himesh Bhandari (037961-24)
Pratik Karna (037977-24)
Sworup Jangam (037975-24)
Mahesh Panta (037987-24)

Submitted to
Mr. Bipes Subedi
Department of Computer Science and Engineering
July 31, 2026

Acknowledgement

We would like to express our sincere gratitude to our instructor for providing us with the opportunity to work on this Database Management System project. Their guidance, feedback, and support throughout the development of the project helped us understand the practical application of database concepts and improve our understanding of relational database design.

We would also like to thank our department and institution for providing the resources and learning environment necessary to complete this project. The knowledge and concepts covered during the course provided the foundation for designing and implementing the Ride-Sharing App database.

Finally, we would like to acknowledge the efforts of all members of our project team. The successful completion of this project was made possible through collaboration, discussion, and shared responsibility in designing the database, developing the schema, writing SQL queries, analyzing normalization, and preparing the final report.

We are grateful to everyone who contributed directly or indirectly to the successful completion of this project.

Abstract

The Ride-Sharing App is a database-driven system designed to manage transportation services. The system connects users who require transportation with drivers who provide rides and maintains structured information about users, drivers, vehicles, trips, payments, complaints, and administrators. The primary purpose of the project is to provide a centralized and reliable database for managing trip records, tracking payments, and handling complaints.

A relational database model was selected because the system contains well-defined entities and relationships that require strong consistency and referential integrity. Primary keys, foreign keys, unique constraints, enumerated values, and triggers are used to maintain data integrity and enforce important business rules. The database is also analyzed using normalization principles from First Normal Form (1NF) through Boyce-Codd Normal Form (BCNF) to minimize redundancy and prevent data anomalies. SQL joins and aggregate functions are used to support operational and analytical queries such as retrieving trip information, calculating driver revenue, and analyzing complaints.

The resulting database provides a structured and consistent foundation for managing ride-sharing operations. The project demonstrates the practical application of important Database Management System concepts, including entity-relationship modeling, relational schema design, keys and relationships, normalization, functional dependencies, integrity constraints, triggers, and SQL-based data analysis.

Table of Contents

Introduction	6
Problem Statement.....	6
Objectives	7
How DBMS Solves the Problem	8
DBMS Concepts Used.....	8
Entity-Relationship Modeling.....	8
Primary Keys and Surrogate Keys	9
Candidate and Alternate Keys	9
Foreign Keys and Referential Integrity.....	9
Cardinality Constraints	9
Normalization	9
Functional Dependencies	9
Enumerated Types	9
Triggers and Integrity Constraints	9
SQL Joins and Aggregate Functions.....	10
Data Model	10
Choice of Data Model: Relational Model	10
Entities in the Data Model	10
Database Schema	11
Normalization	11
First Normal Form	12
Second Normal Form.....	12
Third Normal Form.....	12
Boyce-Codd Normal Form.....	12
Keys and Relationships (ER Diagram).....	12
Relational Schema	14
System Overview.....	17
Sample Database Queries	19
Outcomes.....	19
Screenshots of the System	20
Trip Request / Booking Interface.....	20
Admin Complaint Dashboard	21
Database / SQL Query Output	21
Conclusion.....	22

List of Figures:

Figure 1.1: Entity-Relationship diagram for the Ride-Sharing database.....	13
Figure 2.1: End-to-end trip lifecycle and corresponding database operations.....	18
Figure 3.1: Rider-facing trip request interface.....	20
Figure 3.2: Admin interface for reviewing and resolving complaints.....	21
Figure 3.3: Example output of the revenue-per-driver report.....	21

Introduction

Ride-sharing and shuttle services have become an important part of transportation for students, faculty, and staff within university and their surrounding areas. A ride-sharing system can provide users with a convenient way to request transportation without depending entirely on personal vehicles or fixed public transportation schedules. The Ride-Sharing App is designed as a platform that connects users who require transportation with drivers who provide rides. The system also maintains information about the vehicles used by drivers and records the complete lifecycle of a trip, from the initial request to its completion. In addition to managing trips, the system records payments associated with completed trips and provides a mechanism for users or drivers to submit complaints when an issue occurs. The database is therefore responsible for managing several important entities, including users, drivers, vehicles, trips, payments, complaints, and administrators. These entities are connected through relationships that represent how the system operates. For example, a user can request multiple trips, a driver can complete multiple trips and own multiple vehicles, and each trip can have an associated payment and one or more complaints. A Database Management System (DBMS) is particularly suitable for this application because it provides a structured and reliable way of storing and managing the information. Through primary keys, foreign keys, unique constraints, and other integrity mechanisms, the database can ensure that records remain consistent and that relationships between entities are valid. The main purpose of this project is to design a relational database for the Ride-Sharing App and demonstrate how DBMS concepts can be applied to solve practical problems. The project covers the problem definition, objectives, DBMS concepts, data model, database schema, entity-relationship structure, system workflow, SQL queries, normalization, and implementation outcomes.

Problem Statement

Transportation can become fragmented when students and other members coordinate rides informally through messaging applications or rely on fixed shuttle schedules. Such approaches do not provide a centralized system for maintaining information about trips, payments, drivers, vehicles, and complaints.

The first major problem is the absence of a structured record of trips. Without a centralized database, there is no single source of truth connecting a rider with the driver and vehicle responsible for completing the trip. It becomes difficult to determine who requested a particular trip, which driver completed it, which vehicle was used, and what the final outcome of the trip was.

The second problem is the lack of a reliable payment trail. A ride-sharing system needs to maintain a record of payments associated with individual trips. Without a dedicated payment record, it becomes difficult to verify whether a trip was paid for, determine the amount that was charged, identify the payment method, or audit the transaction at a later time.

The third problem is the lack of an accountability mechanism for complaints. A rider or driver may experience an issue during or after a trip, but without a formal complaint system there is no structured way to record the problem, assign responsibility for handling it, or track its resolution.

These problems demonstrate the need for a centralized and properly structured database. The goal of this project is therefore to design a relational database that provides structured trip management, reliable payment tracking, and an organized complaint-resolution mechanism while maintaining data consistency and minimizing redundancy.

Objectives

The primary objective of this project is to design and implement a normalized relational database for a Ride-Sharing App. The database should accurately represent the entities and relationships required by the application while maintaining consistency and integrity.

The specific objectives of the project are:

1. To design a normalized relational schema for managing users, drivers, vehicles, trips, payments, complaints, and administrators.
2. To correctly represent the cardinality of relationships between entities, such as the one-to-many relationship between drivers and vehicles and the one-to-one relationship between trips and payments.
3. To maintain historically accurate information, particularly the actual amount charged for a trip, even when the value could potentially be calculated from other information.
4. To apply database normalization principles from First Normal Form (1NF) through Boyce-Codd Normal Form (BCNF) and document the functional dependencies associated with each relation.
5. To implement SQL queries that support both operational and analytical requirements, such as retrieving trip details, calculating revenue per driver, and determining the number of complaints handled by each administrator.
6. To maintain referential integrity using primary keys, foreign keys, unique constraints, and appropriate integrity rules.
7. To identify design decisions, trade-offs, and simplifications made during the database design process and discuss possible improvements that could be introduced in a production version of the system.

How DBMS Solves the Problem

A Database Management System directly addresses the problems identified in the Ride-Sharing App by providing structured data storage, integrity constraints, transaction management, and query capabilities.

First, the DBMS provides structured trip records through the trip relation. The trip table contains foreign keys referencing the user, driver, and vehicle involved in the trip. This ensures that every trip is associated with valid records rather than relying on unstructured or manually maintained information. Referential integrity prevents a trip from referencing a user, driver, or vehicle that does not exist in the database.

Second, the DBMS provides a reliable payment trail through the payment relation. Payment information is stored separately from the trip while maintaining a relationship between the two entities. A unique foreign key is used to enforce the intended one-to-one relationship between a trip and its payment record. The payment table also stores a transaction code, payment method, and payment time, allowing individual transactions to be identified and audited.

Third, the DBMS provides an accountability mechanism through the complaints and admin relations. A complaint is associated with a particular trip and assigned to an administrator responsible for reviewing it. The system can therefore maintain information about who filed a complaint, what the complaint was about, which administrator was assigned to it, and whether a response has been provided.

The DBMS also improves consistency when multiple users interact with the system. Transaction mechanisms and ACID properties help ensure that simultaneous operations, such as trip requests, payment records, and complaint submissions, do not result in corrupted or inconsistent data.

Finally, the relational structure allows the system to generate useful reports through SQL. Information such as revenue per driver, complaint counts, average fares, and trip details can be retrieved using joins, aggregate functions, and grouping operations. Therefore, the DBMS does not only store the application's data but also provides the mechanisms required to manage, protect, and analyze that data.

DBMS Concepts Used

Several important Database Management System concepts are applied in the Ride-Sharing App.

Entity-Relationship Modeling

Entity-Relationship modeling is used to identify the main entities within the system, their attributes, and the relationships between them. The primary entities identified in this project are Users, Driver, Vehicles, Trip, Payment, Complaints, and Admin.

Primary Keys and Surrogate Keys

Each relation uses a primary key to uniquely identify individual records. Single-column surrogate keys such as `id` and `trip_id` are used throughout the database. These keys provide a simple and reliable way of identifying records.

Candidate and Alternate Keys

In addition to primary keys, some attributes are uniquely identifiable and therefore act as alternate or candidate keys. For example, `registration_number` in the `Vehicles` relation and `transaction_code` in the `Payment` relation are unique. The `trip_instance` field in `Payment` is also unique and is used to enforce the one-to-one relationship between `Trip` and `Payment`.

Foreign Keys and Referential Integrity

Foreign keys establish relationships between tables and ensure that referenced records exist. For example, `trip.user_id` references `users.id`, while `trip.driver_id` references `driver.id`. Similarly, the vehicle assigned to a trip is referenced through `trip.vehicle_used`.

Cardinality Constraints

Cardinality describes how many records in one entity can be associated with records in another entity. The system includes both one-to-many and one-to-one relationships. For example, one driver can own many vehicles, while each trip has one payment record.

Normalization

Normalization is applied to reduce redundancy and prevent update, insertion, and deletion anomalies. The database is analyzed using First Normal Form, Second Normal Form, Third Normal Form, and Boyce-Codd Normal Form.

Functional Dependencies

Functional dependencies are used to demonstrate the relationships between attributes and keys within each relation. For example, the primary key `users.id` determines the user's name, email address, and phone number.

Enumerated Types

Enumerated types are used for attributes whose values must belong to a predefined set of options. Examples include driver status, vehicle status, trip status, payment method, and the party filing a complaint.

Triggers and Integrity Constraints

A trigger is used to enforce the rule that the vehicle assigned to a trip must belong to the driver assigned to that trip. This is a cross-table business rule that cannot be fully enforced through a simple foreign key alone.

SQL Joins and Aggregate Functions

SQL joins are used to retrieve information from multiple related tables. Aggregate functions such as COUNT, SUM, and AVG are used to produce analytical reports such as the number of complaints handled by an administrator or the total revenue generated by a driver.

Data Model

Choice of Data Model: Relational Model

The relational model was selected as the data model for the Ride-Sharing App. In the relational model, information is organized into tables called relations, with each table representing a particular entity and each row representing an individual record. The relational model is appropriate for this project because the system contains clearly defined entities and well-established relationships. The main entities, including users, drivers, vehicles, trips, payments, complaints, and administrators, have a mostly fixed structure and predictable attributes. One of the main reasons for choosing the relational model is the strong consistency required by the application. A payment must belong to a valid trip, a complaint must reference a valid trip, and an assigned vehicle must belong to a valid driver. Foreign keys, primary keys, unique constraints, and other relational database features can enforce these relationships directly.

Another reason is that the application's data has a well-defined structure. The entities and their attributes are known in advance and do not need the highly flexible schema associated with document-oriented databases. This makes a relational database a natural choice for organizing the information. The application also requires relational and aggregate queries. Questions such as which driver generated the most revenue, how many complaints each administrator handled, and what the average fare is can be efficiently expressed using SQL joins and aggregate functions.

Normalization is another important reason for selecting the relational model. By separating entities into different tables, unnecessary duplication can be avoided. For example, vehicle information is stored separately from driver information because a driver may own multiple vehicles. This prevents repeating vehicle information in a driver's record and makes vehicle-specific data easier to maintain.

Therefore, the relational model provides the consistency, structure, integrity, normalization, and query capabilities required by the Ride-Sharing App.

Entities in the Data Model

The database contains seven primary entities.

Users represent the riders who request and pay for trips.

Driver represents the individuals who operate vehicles and complete trips.

Vehicles represent the vehicles owned by drivers and used to provide rides.

Trip represents an individual ride from the initial request through completion or cancellation.

Payment represents the financial transaction associated with a completed trip.

Complaints represent issues filed by a user or driver concerning a particular trip.

Admin represents the staff members responsible for reviewing and resolving complaints.

Database Schema

The database schema defines the structure of the Ride-Sharing App and specifies how information is stored and connected. The schema contains seven relations: Users, Driver, Vehicles, Trip, Payment, Admin, and Complaints. The users relation stores information about riders. Each user is uniquely identified by an id. The relation also stores the user's name, email address, and phone number. The email address is unique so that two user records cannot use the same email. The driver relation stores information about drivers. Each driver has a unique identifier, name, email address, phone number, and current driver status. The email address is unique, and the driver status indicates whether the driver is available, currently on a trip, or offline. The vehicles relation stores information about vehicles operated by drivers. Each vehicle has a unique identifier, vehicle type, registration number, status, and a foreign key referencing its owner in the Driver relation. The registration number is unique so that every vehicle can be identified independently. The trip relation represents individual rides. It stores the trip identifier, the user who requested the trip, the driver assigned to it, pickup address, destination, price, vehicle used, and trip status. The user, driver, and vehicle fields are connected to their respective relations through foreign keys. The payment relation stores financial information associated with trips. Each payment has a unique identifier and is linked to a trip through trip_instance. Because trip_instance is unique, each trip can have at most one payment record. The relation also stores the transaction code, payment method, and payment timestamp. The admin relation stores information about administrators who manage complaints. Each administrator has a unique identifier, name, and unique email address. The complaints relation stores issues associated with trips. It records the administrator assigned to the complaint, who filed it, the related trip, the complaint description, the response, and timestamps indicating when the complaint was filed and reviewed. In addition to primary and foreign keys, the schema includes a cross-table integrity rule. The driver assigned to a trip must be the owner of the vehicle used for that trip. A trigger is used to verify this condition whenever a trip is inserted or updated.

Normalization

Normalization was applied to the database to reduce redundancy and prevent data anomalies.

First Normal Form

The database satisfies First Normal Form because all attributes contain atomic values. There are no repeating groups or multi-valued attributes. For example, if a driver owns multiple vehicles, each vehicle is stored as a separate record in the Vehicles relation instead of storing a list of vehicles in a single field.

Second Normal Form

The database satisfies Second Normal Form because each relation uses a single-column primary key. Since there are no composite primary keys, partial dependencies on part of a key cannot occur.

Third Normal Form

The database satisfies Third Normal Form with one intentionally discussed exception related to the historical trip price. Normally, non-key attributes should not depend on other non-key attributes. However, the price of a trip is intentionally stored as a historical value because the same route and vehicle can have different prices at different times due to changes in fare rules.

Therefore, storing the actual price on the Trip relation preserves the amount that was charged at the time the trip occurred rather than recalculating it later from changing pricing rules.

Boyce-Codd Normal Form

The schema satisfies BCNF because for each listed non-trivial functional dependency, the determinant is a candidate key. The determinants are either primary keys or documented alternate keys such as vehicle registration numbers, payment trip instances, and transaction codes.

Keys and Relationships (ER Diagram)

The Entity-Relationship diagram represents the entities of the Ride-Sharing database and the relationships between them. The first relationship is between Driver and Vehicles. This is a one-to-many relationship because a driver may own multiple vehicles, while each vehicle belongs to exactly one driver. The second relationship is between Users and Trip. This is also a one-to-many relationship. A user can request multiple trips, while each trip is associated with one user. The Driver and Trip relationship is one-to-many because a driver can complete multiple trips over time, while each individual trip is assigned to one driver. The Vehicles and Trip relationship is also one-to-many. A vehicle can be used for many trips over its lifetime, while each trip uses one vehicle. The Trip and Payment relationship is one-to-one. Each trip is intended to have exactly one payment record. This relationship is enforced using a unique foreign key in the Payment relation. The Trip and Complaints relationship is one-to-many. A single trip may result in zero or more complaints, depending on whether an issue occurs. Finally, the Admin and Complaints relationship is one-to-

many. An administrator may be assigned to multiple complaints, while each complaint is assigned to one administrator.

The ER diagram visually represents all seven entities and these relationships, making the structure of the database easier to understand.

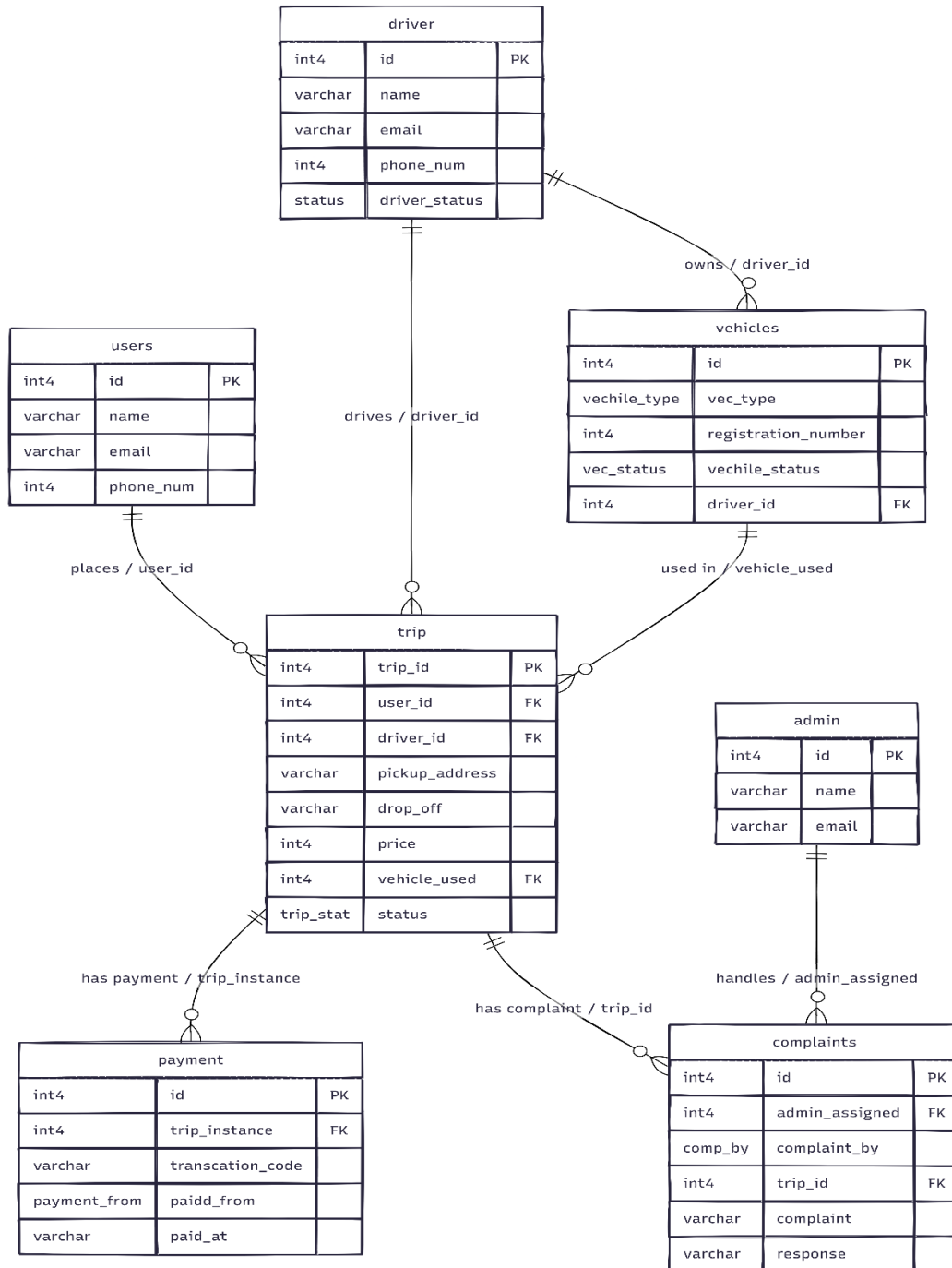


Figure 1.1: Entity-Relationship diagram for the Ride-Sharing database.

Relational Schema

The relational schema translates the conceptual ER model into actual database relations. The **Users** relation contains the details of riders. Its primary key is id, while email is a unique attribute.

The **Driver** relation contains driver information and uses id as its primary key. Driver email addresses are also unique.

The **Vehicles** relation contains vehicle-specific information. Its primary key is id, and registration_number is a unique alternate key. The driver_id foreign key establishes the relationship between each vehicle and its owner.

The **Trip** relation uses trip_id as its primary key. The user_id, driver_id, and vehicle_used fields are foreign keys referencing Users, Driver, and Vehicles respectively.

The **Payment** relation uses id as its primary key. The trip_instance field references trip.trip_id and is unique, enforcing the one-to-one relationship between Trip and Payment. The transaction code is also unique.

The **Admin** relation uses id as its primary key and stores the administrator's name and email address.

The **Complaints** relation uses id as its primary key. It contains foreign keys to Admin and Trip through admin_assigned and trip_id. It also stores the complaint description, response, and relevant timestamps.

Together, these relations provide the complete relational representation of the Ride-Sharing App. The use of primary keys uniquely identifies records, while foreign keys connect related entities and maintain referential integrity.

Schemas code:

```
-- Custom Enum Types
```

```
CREATE TYPE vechile_type AS ENUM ('car', 'bike', 'auto');
```

```
CREATE TYPE vec_status AS ENUM ('active', 'inactive', 'maintenance');
```

```
CREATE TYPE status AS ENUM ('online', 'offline', 'busy');
```

```
CREATE TYPE trip_stat AS ENUM ('requested', 'in_progress', 'completed', 'cancelled');
```

```
CREATE TYPE payment_from AS ENUM ('cash', 'esewa', 'khalti', 'card');
```

```
CREATE TYPE comp_by AS ENUM ('user', 'driver');
```

-- Tables

```
CREATE TABLE users (  
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    name VARCHAR (255) NOT NULL,  
    email VARCHAR (255) UNIQUE NOT NULL,  
    phone_num INT  
);
```

```
CREATE TABLE driver (  
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    name VARCHAR (255) NOT NULL,  
    email VARCHAR (255) UNIQUE NOT NULL,  
    phone_num INT,  
    driver_status status DEFAULT 'offline'  
);
```

```
CREATE TABLE vehicles (  
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    vec_type vechile_type,  
    registration_number INT,  
    vechile_status vec_status,  
    driver_id INT REFERENCES driver(id) ON DELETE SET NULL  
);
```

```
CREATE TABLE admin (  
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    name VARCHAR (255) NOT NULL,
```

```
email VARCHAR (255) UNIQUE NOT NULL  
);
```

```
CREATE TABLE trip (  
    trip_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    user_id INT REFERENCES users(id) ON DELETE CASCADE,  
    driver_id INT REFERENCES driver(id) ON DELETE SET NULL,  
    pickup_address VARCHAR (255),  
    drop_off VARCHAR (255),  
    price INT,  
    vehicle_used INT REFERENCES vehicles(id) ON DELETE SET NULL,  
    status trip_stat DEFAULT 'requested'  
);
```

```
CREATE TABLE payment (  
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    trip_instance INT REFERENCES trip(trip_id) ON DELETE CASCADE,  
    transaction_code VARCHAR (100),  
    paid_from payment_from,  
    paid_at VARCHAR (100)  
);
```

```
CREATE TABLE complaints (  
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    admin_assigned INT REFERENCES admin(id) ON DELETE SET NULL,  
    complaint_by comp_by,  
    trip_id INT REFERENCES trip(trip_id) ON DELETE CASCADE,
```



```
complaint VARCHAR (1000),  
response VARCHAR (1000)  
);
```

The database also contains a trigger that verifies the relationship between a trip's assigned driver and vehicle. Before a trip is inserted or updated, the trigger checks whether the vehicle specified by `vehicle_used` belongs to the driver specified by `driver_id`. If the relationship is invalid, the database raises an exception and prevents the operation.

System Overview

The Ride-Sharing App follows a sequence of database operations that represent the lifecycle of a trip.

The process begins when a user requests a trip. At this stage, a new record is created in the Trip relation containing the user and the requested trip information.

After the request is created, a driver and vehicle are assigned to the trip. The corresponding trip record is updated with the assigned driver and vehicle. The integrity rule ensures that the selected vehicle belongs to the assigned driver.

The trip then enters an ongoing state while the driver provides the ride. Once the ride is completed, the trip status is changed to completed.

After completion, a payment record is created in the Payment relation. This record stores the trip associated with the payment, the transaction code, the payment method, and the time at which payment was made.

Following payment, the system checks whether an issue was reported. If there is no complaint, the trip can be archived as a completed ride. If an issue is reported, a complaint record is created and linked to the corresponding trip.

The complaint is then assigned to an administrator. The administrator reviews the complaint and provides a response or resolution. The complaint record can subsequently be updated with the response and review timestamp.

This lifecycle demonstrates how the different database entities work together to support the complete ride-sharing process.

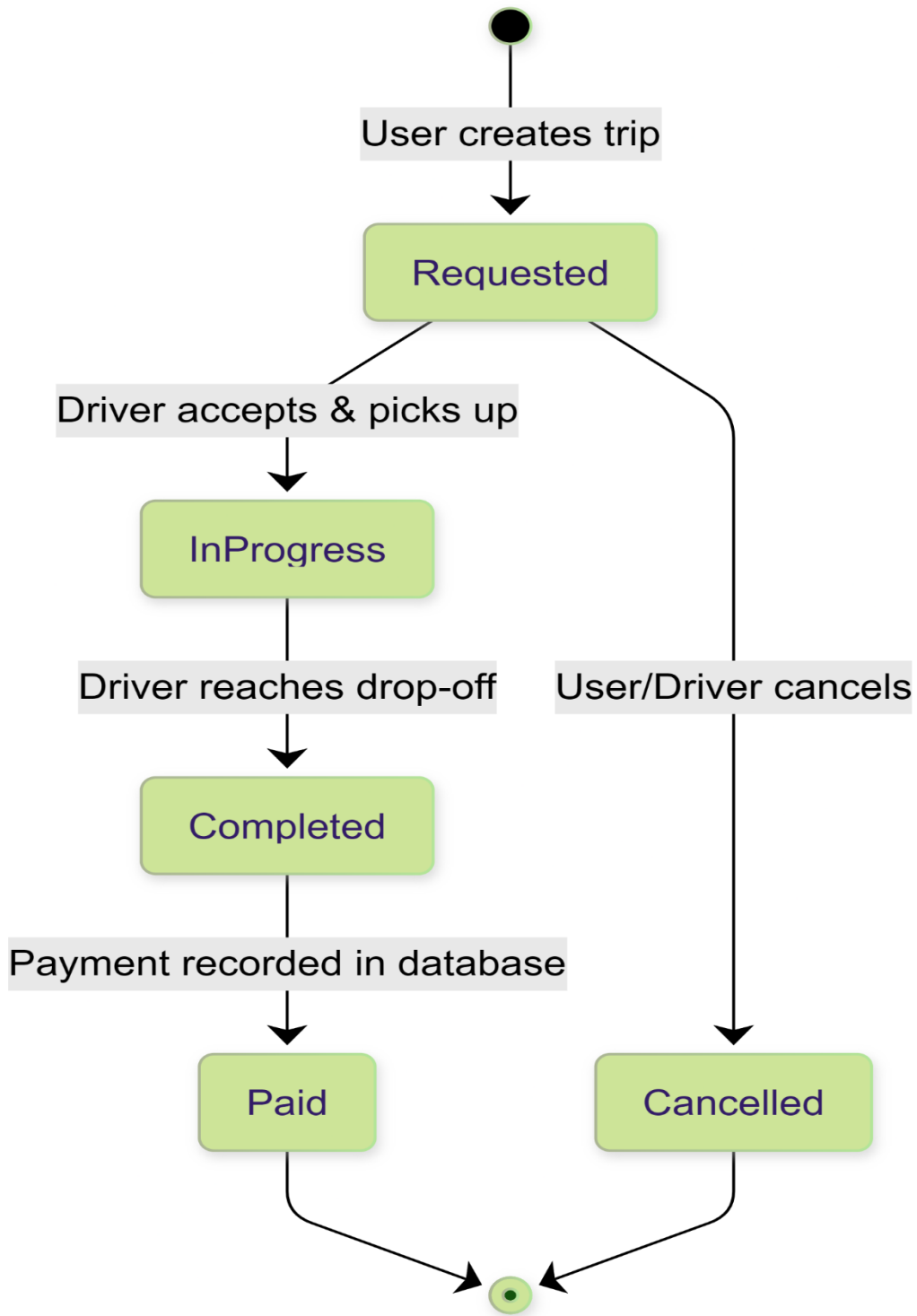


Figure 2.1: End-to-end trip lifecycle and corresponding database operations.

Sample Database Queries

The database also supports operational and analytical queries. For example, a trip-detail query can join Users, Driver, and Vehicles to display the rider, driver, vehicle registration number, pickup location, destination, price, and status of each trip.

A complaints-per-admin query can count the number of complaints assigned to each administrator and determine how many have received responses.

A revenue-per-driver query can calculate the total number of completed trips, total revenue, and average fare for each driver.

A data-integrity query can also identify cases where the driver assigned to a trip does not match the owner of the vehicle used in that trip.

These queries demonstrate that the database is capable of supporting both day-to-day operational requirements and higher-level reporting and analysis.

Outcomes

The project resulted in a structured relational database for the Ride-Sharing App containing seven major entities and the relationships required to represent the system.

The database design provides a normalized structure covering Users, Driver, Vehicles, Trip, Payment, Complaints, and Admin. The relationships between these entities are represented using primary keys and foreign keys, while unique constraints are used for attributes that must not be duplicated.

Referential integrity is maintained throughout the database. Foreign keys prevent records from referencing entities that do not exist, while the unique constraint on the payment's trip reference enforces the intended one-to-one relationship between trips and payments.

The project also demonstrates the application of normalization principles from 1NF through BCNF. Functional dependencies were identified for the relations, and the schema was designed to minimize redundancy and prevent common database anomalies.

The database supports SQL queries for operational and analytical requirements. These include retrieving complete trip information, determining the number of complaints handled by each administrator, calculating revenue per driver, and checking for inconsistencies between assigned drivers and vehicles.

During the design process, several data-quality issues were also identified and corrected. These included incorrect column names and inappropriate data types, such as storing price as an integer

instead of a decimal value and storing payment timestamps as text rather than an appropriate date/time type.

The project also documents design trade-offs. In particular, the Admin-to-Complaint relationship is modeled as one-to-many rather than many-to-many. This simplifies the current design while leaving the possibility of supporting multiple administrators or review panels in a future version.

Overall, the resulting database provides a consistent foundation for managing ride-sharing operations while demonstrating important DBMS concepts such as relational modeling, normalization, referential integrity, constraints, triggers, joins, and aggregate queries.

Screenshots of the System

This section provides visual evidence of the implemented Ride-Sharing database and its associated system components.

Trip Request / Booking Interface

The first screenshot demonstrates the interface through which a user can request or book a trip. It represents the user-facing portion of the ride-sharing process and corresponds to the creation of a trip record in the database.

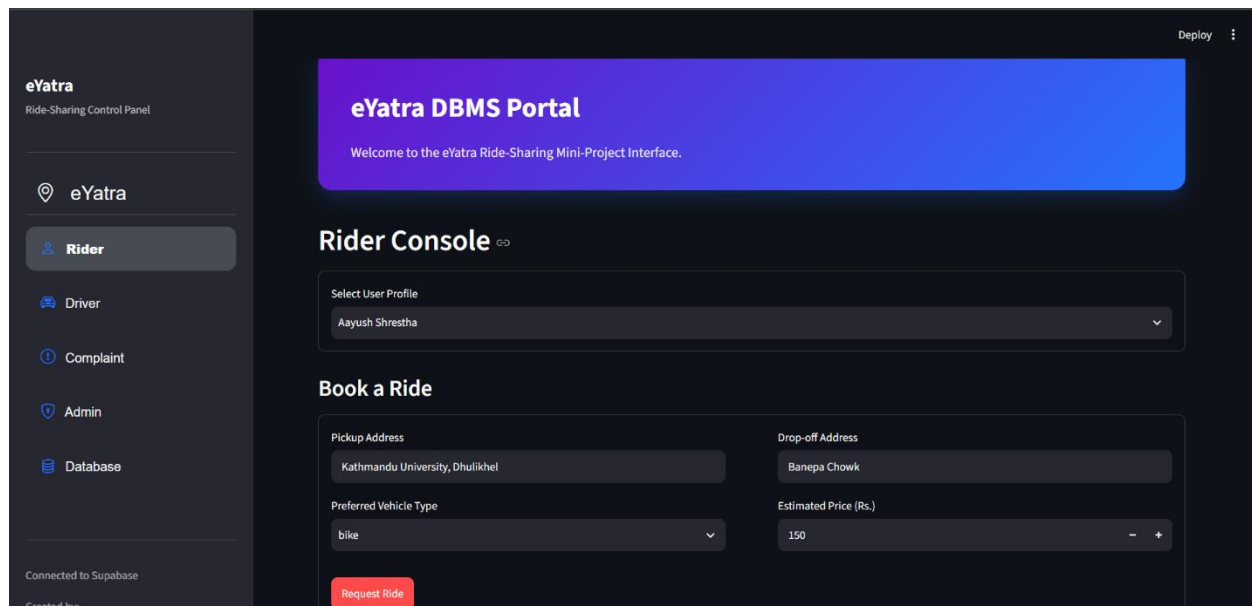


Figure 3.1: Rider-facing trip request interface.

Admin Complaint Dashboard

The second screenshot demonstrates the administrator's view for managing complaints. The interface represents the complaint-management process in which an administrator can review issues associated with trips and provide an appropriate response or resolution.

The screenshot shows a web interface titled "Resolve Pending Customer Complaints". It features a form with the following elements:

- A dropdown menu labeled "Select Complaint ID to Address" with the value "13" selected.
- A green bar displaying "By: Passenger | Trip ID: 204".
- A blue bar displaying "Description: Asked for more money".
- A dropdown menu labeled "Assign Resolving Admin" with the value "Sandeep Sharma" selected.
- A text area labeled "Write official resolution response" containing the text "Fired".
- A red button labeled "Submit Official Response".

Figure 3.2: Admin interface for reviewing and resolving complaints.

Database / SQL Query Output

The third screenshot demonstrates the database implementation and query results. An example can show the output of the revenue-per-driver query, which retrieves the number of completed trips, total revenue, and average fare for each driver.

The screenshot shows a dashboard titled "Trip Transaction History" with a navigation bar (Users, Drivers, Vehicles, Trips, Payments, Complaints, Admins) and a "Deploy" button. The dashboard displays three summary cards: "Total Bookings: 19", "Completed Rides: 11", and "Average Trip Fare: Rs. 194.74". Below these cards is a table with the following data:

	trip_id	user_id	driver_id	pickup_address	drop_off	price	vehicle_used	status
0	101	1	1	Dhulikhel, KU Main Gate	Banepa, Chardobato	150	1	completed
1	102	2	2	Koteshwor, Kathmandu	Kupandole, Lalitpur	350	2	completed
2	104	4	1	Pulchowk, Lalitpur	Jhamsikhel	120	1	completed
3	105	6	5	Chabahil, Kathmandu	Gaushala Mitrapark	100	5	completed
4	106	7	6	Lagankhel, Lalitpur	New Baneshwor	250	6	completed
5	107	8	7	Kalanki, Kathmandu	Balaju Bypass	300	7	ongoing
6	108	9	8	Bhaktapur Durbar Square	Tinkune Koteshwor	450	8	completed
7	109	10	6	Tripatheshwor, Kathmandu	Thapathali Height	80	6	completed
8	110	11	5	Jawalakhel, Lalitpur	Satdobato Chowk	120	5	completed
9	111	12	8	Narayan Gopal Chowk	Budhanilkantha Temple	280	8	ongoing

Figure 3.3: Example output of the revenue-per-driver report.

The screenshots provide practical evidence that the database design can be connected to system operations and that the implemented database can store, retrieve, and analyze the information required by the Ride-Sharing App.

Conclusion

This project designed and documented a relational database for a Ride-Sharing App. The database models the main entities involved in the system, including users, drivers, vehicles, trips, payments, complaints, and administrators, while maintaining appropriate relationships between them.

The relational model was selected because the application requires strong consistency, a well-defined schema, referential integrity, and SQL-based analytical queries. Primary keys, foreign keys, unique constraints, and a trigger were used to enforce the required integrity rules and ensure that related records remain consistent.

The database was also analyzed using normalization principles from First Normal Form through Boyce-Codd Normal Form. The normalization process reduces redundancy and prevents common data anomalies, while the intentional storage of the historical trip price preserves the amount that was actually charged at the time of a ride.

The implemented SQL queries demonstrate that the database can support both operational and analytical tasks, including retrieving trip details, tracking payments, analyzing complaints, calculating driver revenue, and checking data integrity.

Overall, the Ride-Sharing database provides a structured, normalized, and consistent foundation for managing transportation data while demonstrating the practical application of fundamental Database Management System concepts.

References

A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2019.

R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.

PostgreSQL Global Development Group, *PostgreSQL Documentation*. Available: <https://www.postgresql.org/docs/>

W3Schools, *SQL Tutorial*. Available: <https://www.w3schools.com/sql/>

Microsoft, *Database Normalization Description*. Microsoft Support. Available: <https://support.microsoft.com/>

Oracle, *MySQL 8.0 Reference Manual: SQL Statements*. Oracle Corporation. Available: <https://dev.mysql.com/doc/refman/8.0/en/>